

```
import pandas as pd
df=pd.read_csv('/content/Symptom2Disease.csv')
display(df)
```

	Unnamed: 0	label	text	
0	0	Psoriasis	I have been experiencing a skin rash on my arm...	 
1	1	Psoriasis	My skin has been peeling, especially on my kne...	
2	2	Psoriasis	I have been experiencing joint pain in my fing...	
3	3	Psoriasis	There is a silver like dusting on my skin, esp...	
4	4	Psoriasis	My nails have small dents or pits in them, and...	
...	...	...	...	
1195	295	diabetes	I'm shaking and trembling all over. I've lost ...	
1196	296	diabetes	Particularly in the crevices of my skin, I hav...	
1197	297	diabetes	I regularly experience these intense urges and...	
1198	298	diabetes	I have trouble breathing, especially outside. ...	
1199	299	diabetes	I constantly sneeze and have a dry cough. My i...	

1200 rows × 3 columns

Next steps:

[Generate code with df](#)

[New interactive sheet](#)

```
df = df.dropna()
display(df.head())
display(df.isnull().sum())
df.info()
df['label'].value_counts()
```

	Unnamed: 0	label	text
0	0	Psoriasis	I have been experiencing a skin rash on my arm...
1	1	Psoriasis	My skin has been peeling, especially on my kne...
2	2	Psoriasis	I have been experiencing joint pain in my fing...
3	3	Psoriasis	There is a silver like dusting on my skin, esp...
4	4	Psoriasis	My nails have small dents or pits in them, and...
0			

Unnamed: 0	0
label	0
text	0

dtype: int64
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1200 entries, 0 to 1199
Data columns (total 3 columns):
Column Non-Null Count Dtype
--- ---
0 Unnamed: 0 1200 non-null int64
1 label 1200 non-null object
2 text 1200 non-null object
dtypes: int64(1), object(2)
memory usage: 28.3+ KB

	count
label	
Psoriasis	50
Varicose Veins	50
Typhoid	50
Chicken pox	50
Impetigo	50
Dengue	50
Fungal infection	50
Common Cold	50
Pneumonia	50
Dimorphic Hemorrhoids	50
Arthritis	50
Acne	50
Bronchial Asthma	50
Hypertension	50
Migraine	50
Cervical spondylosis	50
Jaundice	50
Malaria	50
urinary tract infection	50
allergy	50
gastroesophageal reflux disease	50
drug reaction	50
peptic ulcer disease	50
diabetes	50

dtype: int64

```
import nltk
import re
from nltk.corpus import stopwords
```

```
from nltk.stem import WordNetLemmatizer

nltk.download('stopwords')
nltk.download('wordnet')

stop_words = set(stopwords.words('english'))
lemmatizer = WordNetLemmatizer()

def clean_text(text):
    text = text.lower()
    text = re.sub(r'^a-zA-Z ', '', text)
    words = text.split()
    words = [lemmatizer.lemmatize(word) for word in words if word not in stop_words]
    return " ".join(words)

df['clean_text'] = df['text'].apply(clean_text)
```

[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Unzipping corpora/stopwords.zip.
[nltk_data] Downloading package wordnet to /root/nltk_data...

```
from sklearn.feature_extraction.text import TfidfVectorizer

vectorizer = TfidfVectorizer(max_features=5000)
X = vectorizer.fit_transform(df['clean_text'])
y = df['label']
```

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
```

```
from sklearn.svm import LinearSVC

model = LinearSVC()
model.fit(X_train, y_train)
```

▼ LinearSVC ⓘ ?

LinearSVC()

```
from sklearn.metrics import accuracy_score, classification_report

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

Accuracy: 0.9708333333333333				
	precision	recall	f1-score	support
Acne	1.00	1.00	1.00	10
Arthritis	1.00	1.00	1.00	10
Bronchial Asthma	1.00	1.00	1.00	10
Cervical spondylosis	1.00	1.00	1.00	10
Chicken pox	0.91	1.00	0.95	10
Common Cold	1.00	1.00	1.00	10
Dengue	1.00	0.90	0.95	10
Dimorphic Hemorrhoids	1.00	1.00	1.00	10
Fungal infection	1.00	1.00	1.00	10
Hypertension	1.00	1.00	1.00	10
Impetigo	1.00	1.00	1.00	10
Jaundice	1.00	1.00	1.00	10
Malaria	1.00	1.00	1.00	10
Migraine	1.00	0.90	0.95	10
Pneumonia	0.83	1.00	0.91	10
Psoriasis	1.00	1.00	1.00	10
Typhoid	1.00	1.00	1.00	10
Varicose Veins	1.00	1.00	1.00	10
allergy	1.00	1.00	1.00	10
diabetes	0.82	0.90	0.86	10
drug reaction	0.86	0.60	0.71	10
gastroesophageal reflux disease	0.91	1.00	0.95	10
peptic ulcer disease	1.00	1.00	1.00	10
urinary tract infection	1.00	1.00	1.00	10
accuracy			0.97	240

macro avg	0.97	0.97	0.97	240
weighted avg	0.97	0.97	0.97	240

```
def predict_disease(symptom_input):
    cleaned = clean_text(symptom_input)
    vector = vectorizer.transform([cleaned])
    prediction = model.predict(vector)
    return prediction[0]
```

```
sample_symptom = "I have fever and severe headache"
predicted_disease = predict_disease(sample_symptom)
print(f"For the symptoms: '{sample_symptom}', the predicted disease is: {predicted_disease}")
```

For the symptoms: 'I have fever and severe headache', the predicted disease is: Malaria

```
sample_symptom = "I have fever and severe headache"
cleaned_symptom = clean_text(sample_symptom)
vector = vectorizer.transform([cleaned_symptom])
decision_scores = model.decision_function(vector)
print(decision_scores)
```

```
[[-1.00292814 -1.24317411 -0.86054664 -1.47994795 -0.6417235 -1.51359567
 -0.72925034 -1.28753119 -1.33350999 -0.76398248 -0.95749371 -1.0351057
 -0.28763378 -0.87397843 -1.53292573 -1.32356471 -0.83453151 -0.90886609
 -1.07943512 -1.33317414 -0.84724793 -1.47166975 -1.28405973 -1.05105706]]
```

```
import numpy as np

def predict_with_confidence(symptom_input):
    cleaned = clean_text(symptom_input)
    vector = vectorizer.transform([cleaned])

    prediction = model.predict(vector)[0]
    scores = model.decision_function(vector)

    predicted_class_index = list(model.classes_).index(prediction)
    confidence = scores[0][predicted_class_index]

    # Convert to readable percentage
    confidence_percent = round((1 / (1 + np.exp(-confidence))) * 100, 2)

    return prediction, confidence_percent

# ===== CHATBOT =====
print("🤖 Medical Chatbot is Ready!")
print("Enter your symptoms (type 'exit' to stop)\n")

while True:
    user_input = input("Enter symptoms: ")

    if user_input.lower() == "exit":
        print("👋 Thank you for using the Medical Chatbot.")
        break

    disease, confidence_score = predict_with_confidence(user_input)

    print("\n🔗 Predicted Disease:", disease)
    print("📊 Confidence:", confidence_score, "%")
    print("-" * 50)
```